

Админское руководство к Венгерскому Шкафу

Содержание

- [Вместо введения от Андрея](#)
- [Немного теории и сами цели](#)
- [Готовим сервер](#)
 - [Блокируем сканеры](#)
 - [Устанавливаем Docker](#)
 - [Ядро AmneziaWG \(опционально\)](#)
 - [Разгоняем скорости трафика](#)
- [Поднимаем VPN](#)
 - [Про веб-панели](#)
 - [3x-ui — выходной нод](#)
 - [О входных нодах](#)
 - [3x-ui — входной нод](#)
 - [wg-easy / AmneziaWG](#)
 - [NaiveProxy](#)
 - [Mieru](#)
 - [Telemt](#)
- [Генерация конфигов \(в разработке\)](#)

Вместо введения от Андрея

В данном "гайде" я распишу свой путь по поднятию ВПН и прокси сервисов на своих серверах от небольшой теории и подготовки серверов, до развертывания нескольких сервисов с детальной настройкой

Важно! Это не пошаговый гайд, не истина в последней инстанции — скорее описание моего пути, который в целом может повторить, если не каждый, то почти каждый человек, разбирающийся в серверах, Linux, Docker и сетях (хотя бы немного). Не всё, что хотелось сделать, удалось реализовать, но постарался максимально приблизиться к целям, которые сам себе ставил. Также всё будет описано только для IPv4 как для подавляющего в России транспорта. Не все протоколы нормально его поддерживают, некоторые требуют допила напильником... Да и это всё про «сохранение хоть какого-то доступа в интернет». Так что если кому-то важен IPv6 — я вас очень уважаю, но уже сами, котаны, сами ^_^

Также все пояснения и причины тех или иных решений основаны на наблюдениях сообщества, а не точных данных, которых увы ни у кого нет. Я не даю везде подробные команды — предполагаем, что базовые навыки работы с Linux, SSH и Docker у человека есть. В противном случае этот гайд бы состоял на 70% не из конфигов и их описаний, а из того «как мне запустить Docker»

Немного теории и сами цели

Цели следующие:

1. Возможность поднять достаточно быстро на арендованном сервере несколько разных VPN протоколов (Частично)
 - XRAY Vless (и все протоколы доступные в xray-core). Сделано
 - AmneziaWG (1.5 и 2.0). Сделано
 - NaiveProxy. Сделано
 - Mieru. В процессе. Сервер поднимается, нет поддержки в hc-box, нет генерации конфигов для клиентов
2. Управление пользователями на сервере в веб панелях ну или по крайней мере удобным конфигом (Сделано)
3. Возможность поднимать мультихоп сервера (Сделано, с оговорками)
4. Возможность генерировать конфиги для общедоступных клиентов, желательно с возможностью подписок (Сделано, с оговорками)
5. Блокировка нежелательного трафика на сервере (Пока не сделано)

Про протоколы, управление пользователями и генерацию конфигов будут отдельные разделы далее. Про сервера и мультихоп надо рассказать чуть больше

ВПН сервер должен быть арендован за границей. Приоритет у стран со "свободным" интернетом и без каких то явных ограничений. Основные направления для выбора: Сербия, Финляндия, Эстония, Польша, Нидерланды. Франция и прочие развитые страны ЕС рассматриваем с меньшим рвением, они с ВПН тоже достаточно активно борются, как и с нелегальным контентом, к которым могут относиться всяческие онлайн кинотеатры. Это касается точки выхода

В случае однохоповой системы это будет единственный сервер, и на нём мы и остановимся. В случае мультихоп систем мы выстраиваем цепочку из серверов. Обычно это два-три сервера. Входная точка в РФ. Трафик внутри страны меньше модерируется, плюс есть вероятность в случае введения оплаты заграничного трафика - обойти эти ограничения. Кроме этого, РКН активно палит IP адреса серверов узнавая IP выхода.. В случае мультихоп системы блокировка IP выхода с меньшей вероятностью повлияет на работоспособность системы, так как блокировка "обычно" затрагивает пользователей, но не общение "сервер - сервер". Полный «фарш» — это трипл-хоп система: Сервер РФ → Сервер в СНГ → Сервер за бугром

Частично эффекта можно добиться 2мя IP на одном из серверов. Например вход в РФ с одним IP и выход за бугром с 2мя IP может частично выполнять роль трипл хопа. Трафик идет на сервер в РФ, оттуда на первый IP выходного сервера, а выход через второй IP. В таком случае если РКН и узнает точку выхода, то ничего уже не сможет сделать (наверное)

Для поднятия всего софта на серверах будет использоваться Docker. В качестве системы рекомендуется Ubuntu 24.04. Шаблоны конфигов, скриптов и структур папок будут лежать по отдельной ссылке; за доступом можно обратиться ко мне в личку или в наш чатик

И да, мы не будем постоянно проговаривать все команды — только основные. В целом почти всё, что я сделал, достаточно просто гуглится и много где расписано. Команды, которые будут приводиться, — для Ubuntu. Излишней идеей безопасности я не страдаю, потому команды лично я выполняю от root. Да неправильно, да небезопасно... но это VPN-сервера, на которых ничего кроме трафика не бежит —

потому хрен с ним. Если хочется — заморачивайтесь и к командам добавляйте **sudo**. **Обязательно:** запретите вход по паролю и включите авторизацию по SSH-ключу — это минимум, который стоит сделать даже на «одноразовых» VPN-хостах

Готовим сервер (любой, хоть входной, хоть выходной)

Во-первых, обновляем софт на сервере:

```
apt update
apt upgrade -y
```

Блокируем сканеры

РКН активно сканирует все подозрительные подключения. Для этого используется целый ряд серверов и организаций. Занимаются как активным пробингом сайтов, так и сканированием портов на предмет ответов выдающих на порту впн. Если блокировать такой трафик, то вероятность продлить жизнь сервера без "вас обнаружили" моментов такой трафик лучше блокировать

Есть два варианта:

1. Установить вот эту штуку на сервер: <https://github.com/dotX12/traffic-guard>
2. Использовать мои скрипты из папки **scanners_block**

Если идём вторым путём:

1. Устанавливаем нужные утилиты:

```
apt install jq curl ipset -y
```

2. Копируем содержимое папки в **/etc/**
3. Делаем нужные файлы исполняемыми:

```
chmod +x /etc/ipset-tools/block-input/run.sh
chmod +x /etc/ipset-tools/lists-update/run.sh
```

4. Обновляем список системных служб и включаем нужные сервисы:

```
systemctl daemon-reload
systemctl enable --now ipset-update.timer
systemctl enable --now scanners-block.timer
```

5. После можно посмотреть список ipset и статусы служб:

```
systemctl status ipset-update.service
systemctl status scanners-block.service
ipset list -n
ipset list scanners-block-list-v4
```

Если всё сделано правильно, то сканеры вас тревожить будут несколько меньше. Рекомендуемые списки сканеров:

```
https://raw.githubusercontent.com/tread-
lightly/CyberOK_Skipa_ips/refs/heads/main/lists/skipa_cidr.txt
https://raw.githubusercontent.com/shadow-netlab/traffic-guard-
lists/refs/heads/main/public/antiscanner.list
https://raw.githubusercontent.com/shadow-netlab/traffic-guard-
lists/refs/heads/main/public/government_networks.list
https://raw.githubusercontent.com/shadow-netlab/traffic-guard-
lists/refs/heads/main/public/skipa.list
```

В моём варианте они хранятся в конфиге `/etc/ipset-tools/lists-update/config.json`

Устанавливаем Docker

Нам нужен будет docker и docker compose. Потому устанавливаем их официальным скриптом:

```
curl -fsSL https://get.docker.com -o install-docker.sh
sh install-docker.sh
```

Устанавливаем ядро амнезии (опционально)

Если планируем хостить AmneziaWG, то нам нужно установить модуль ядра амнезии. Без неё увы использовать тот контейнер, который был выбран не получится. При этом если бы и получилось - протокол был бы очень "требовательным" к железу, если будет исполняться в userspace

Устанавливаем по этой [инструкции](#)

Разгоняем скорости трафика и прочее

Добавляем в конец файла `/etc/sysctl.conf` строчки

```
net.ipv4.conf.all.src_valid_mark=1
net.ipv4.ip_forward = 1

net.core.default_qdisc=fq
net.ipv4.tcp_congestion_control=bbr
```

Применяем:

```
sysctl -p
```

Поднимаем ВПНы

Про веб панели

У трёх инструментов будет своя веб панель для управления. Их можно вытащить наружу, выдать сертификат, иметь доступ напрямую... Но это палевно. В целом наличие веб панели на хосте это как красная тряпка для РКН, ТСПУ, провайдеров и даже самих хостеров. Некоторые панели, такие как 3x-ui вообще не вызывают к себе любви со стороны хостеров если обнаружены

Потому рекомендуемый сценарий: доступ к веб-панелям по SSH-туннелю. В дальнейшем все настройки будут показаны исходя из этого. Для использования веб-панельки через SSH-туннель:

```
ssh -L <PORT>:localhost:<PORT> root@<ip>
```

Где порт это порт вашей веб панели. После чего она станет доступна у вас локально по адресу 127.0.0.1:PORT

У всех веб панелей постоянно меняется визуал, оформление и делать скриншоты смысла никакого нет. Я буду описывать суть, а дальше думаю разберётесь

Дальше будет приводиться настройка и пояснения для сервисов общего докер компоуз файла. В рамках входной ноды все сервисы будут подниматься в рамках одного сетевого стека и выходить в интернет не напрямую, а через туннель к следующему хопу. Если система - сингл хоп, то отличие будет только в отсутствии дополнительной маршрутизации

3x-ui (выходной нод при мультихопе)

[Ссылка на проект](#)

Копируем на сервер папку `configs_and_scripts/out_hop/hungcabinet` в папку `/etc/hungcabinet`. Она содержит базово настроенную 3x-ui панель с само подписанными сертификатами. Рекомендованный мной докер файл такой:

```
services: #NUXT NL L
  3x-ui:
    image: ghcr.io/mhsanaei/3x-ui:latest
    container_name: 3x-ui
    hostname: 3x-ui
    ports:
      - "4696:4696"          #xray port, нужно прописать свой при создании
                             входящего подключения
      - "127.0.0.1:2053:2053" #xray panel
    volumes:
      - ./x-ui/db:/etc/x-ui
```

```

- ./x-ui/cert:/root/cert
cap_add:
  - NET_ADMIN
  - SYS_MODULE
devices:
  - /dev/net/tun:/dev/net/tun
sysctls:
  - net.ipv4.conf.all.src_valid_mark=1
  - net.ipv4.ip_forward=1
environment:
  XRAY_VMESS_AEAD_FORCED: "false"
  XUI_ENABLE_FAIL2BAN: "true"
tty: true
networks:
  main:
    ipv4_address: 172.128.0.2
restart: unless-stopped
networks:
  main:
    driver: bridge
    ipam:
      config:
        - subnet: 172.128.0.0/24

```

Далее всё просто:

```

cd /etc/hungcabinet
docker compose up -d

```

Когда сервис поднимется, с помощью SSH-туннеля цепляемся к <https://127.0.0.1:2053> и настраиваем входящее подключение. К нему будет подключаться входная нода.

Безопасность: пароль по умолчанию `admin/admin` — смените сразу после первого входа. Не оставляйте его на проде.

Если вдруг самоподписанный сертификат протухнет, можно сделать:

```

cd /etc/hungcabinet/x-ui/cert/
chmod +x ./generate.sh
./generate.sh

```

Это сгенерирует новые самоподписанные сертификаты, для работы через локальный порт других и не нужно

Возможно на момент написания документации будет актуально что то другое, но рекомендуется в качестве входного подключения выбрать vless + reality

1. Переходим в панели на "Входящие"

2. Протокол vless
3. Порт выбираете сами, его надо будет прописать в портах докер компоуз файла
4. Протокол оставляем без изменений
5. Поток RAW
6. Безопасность Reality, обязательно убедиться, что сгенерировались приватный и публичный ключик
7. uTLS ставим random
8. Целевой sni берём какой то подходящий локации, но в целом для выходной ноды как показывает практика это не так критично. Можно взять www.nvidia.com или что то, что предлагает сама панель
9. Жмём создать

Теперь создадим клиента (он же пользователь). Для выходного нода достаточно одного

1. Идём в "Клиенты"
2. Жмём "Добавить клиентов"
3. В появившемся окне в "привязанные входящие" выбираем наш инбоунд. Он будет один, но опознать можно будет по порту
4. После прикрепления появится поле Flow — ставим `xtls-rprx-vision`
5. Можно добавить имя пользователя, удобочитаемое как в поле email, так и в комментарий. В email пишем просто последовательность латинских букв — это не обязательно электронная почта
6. Жмём «Создать»

После этого клиент появится в списке, у него будет иконочка qr кода, если нажать на неё, то можно получить vless ссылку на подключение этого клиента. Важно, нам нужна не ссылка на подписку! Нам нужна ссылка VLESS, это как правило второй qr, иногда в скрытом под "спойлером" элементе. Копируем её. После чего надо доработать её напильником, указав вместо `localhost:<ваш внутренний докер порт>` данные о вашем сервере `<ваш внешний ip>:<ваш внешний докер порт>`

Должно получиться что то типа

```
vless://8510aaec-0695-4fdf-ae03-0501ed8c343f@2.3.4.5:678?encryption=...
```

ВАЖНО!!! (из личного опыта) При генерации vless подключения очень часто генерируются короткие short ids. Обычно это вкладка Безопасность. Короткие - это меньше 6 символов, такие лучше удалять! Иначе ссылки могут быть битые и клиенты не смогут подключиться

Нам эта ссылка ещё пригодится — сохраняем её

Если порт не был прописан заранее, прописываем порт в докер компоуз и перезапускаем стэк:

```
docker compose down
docker compose up -d
```

Схема мультихопа (дабл-хоп):

```
flowchart LR
    Client[Клиент] --> EntryRU[Вход РФ]
    EntryRU --> ExitEU[Выход EU]
    ExitEU --> Internet[Интернет]
```

О входной нодах

Далее будут приведены примеры настройки разных протоколов на входной ноде, но возможно вам нужны будут не все протоколы, потому в каждом разделе о конкретном протоколе будут указаны те части сервиса, которые непосредственно относятся к протоколу. Эти сервисы в случае необходимости можно убрать из итогового докер компоуз файла, если вам протокол не нужен

Что касается 3x-ui в данной схеме. Его убрать не получится, так как он обеспечивает связь с выходной нодой. Все примеры будут работать и без таковой, но описано всё будет для ситуации когда у нас мультихоп система

Также в выбранном сценарии есть особенность сетевых настроек. Все сервисы в данном стеке делят один общий сетевой стек, потому все настройки сети производятся в рамках главного сервиса, а именно 3x-ui, потому порты и сетевые разрешения указываются именно в этом сервисе, а не в своих секциях. Потому порты всех сервисов будут указаны в основном 3x-ui

3x-ui (входной нод)

[Ссылка на проект](#)

По сути тут особой разницы нет с тем, как мы поднимали входную ноду. Часть докер компоуз файла такая же. Разница только в том, что нам важно указать IP адрес самого 3x-ui контейнера И крайне важно правильно настроить порты и сеть. В примерах и в папке будет конфиг для ситуации, когда у вас есть "выходной сервер". В таком режиме почти все сервисы поднимаются в рамках одного сетевого стека. Ведущим является контейнер 3x-ui и выход во второй нод обеспечивается через него. Если система сингл хоп, то каждый сервис может быть поднят в своем сетевом стеке, так будет на порядок чище. В таком случае порты сервисов нужно указывать в самих сервисах. В целом никто не мешает поднять всё аналогично дабл хопу и в случае одного хопа, проблем в этом тоже не будет. Порты указаны чисто ради примера, используйте свои, которые будут настроены в конечных сервисах

Общая архитектура дабл хопа такая:

1. 3x-ui обеспечивает связь с выходной нодой (или следующей нодой в цепочке, если у вас их три) и маршрутизирует в него трафик
2. 3x-ui поднимает vless (возможно и другие) протоколы для подключения vpn клиентов пользователей
3. 3x-ui поднимает tun интерфейс для работы остальных протоколов, который сам не может обеспечить
4. 3x-ui поднимает socks интерфейс для работы протоколов, клиентам в которых предпочтительнее сокс выход, а не напрямую (такие имеются)

```
3x-ui:
  image: ghcr.io/mhsanaei/3x-ui:latest
```

```

container_name: 3x-ui
hostname: 3x-ui
ports:
  - "1111:1111"          #xray port если вам нужен будет таковой
  - "127.0.0.1:2053:2053" #xray panel - порт веб панели
  - "1112:1112/udp"      #awg port - только если поднимаем amnezia
  - "127.0.0.1:51821:51821/tcp" #awg panel - веб панель амнезии, тоже
указываем если надо
  - "1113:1113"          #telemt port - если поднимаем telemt mtproto
  - "127.0.0.1:9090:9090" #telemt metrics - если поднимаем telemt
mtproto
  - "127.0.0.1:9091:9091" #telemt api - если поднимаем telemt mtproto
  - "127.0.0.1:8080:8080" #telemt panel - если поднимаем telemt mtproto
  - "80:80"               #naive получение сертификатов
  - "443:443"             #naive https - если поднимаем naiveпроху, по
возможности использовать именно 443
  - "443:443/udp"        #naive quic - если поднимаем naiveпроху в quic
режиме, по возможности использовать именно 443
volumes:
  - ./x-ui/db:/etc/x-ui
  - ./x-ui/cert:/root/cert
  - ./x-ui/scripts:/root/scripts
cap_add:
  - NET_ADMIN
  - NET_RAW
  - SYS_MODULE
devices:
  - /dev/net/tun:/dev/net/tun
sysctls:
  - net.ipv4.conf.all.src_valid_mark=1
  - net.ipv4.ip_forward=1
environment:
  XRAY_VMESS_AEAD_FORCED: "false"
  XUI_ENABLE_FAIL2BAN: "true"
tty: true
# своего хелсчека у 3x-ui нет, но нам нужно выждать после его поднятия какое
то время, чтоб поднялись инбаунды
# потому делаем вот такой хак/воркэраунд, все зависимые сервисы будут ждать
поднятия xray
healthcheck:
  test: ["CMD", "true"]
  interval: 1s
  timeout: 1s
  retries: 1
  start_period: 5s
networks:
  main:
    ipv4_address: 172.128.0.2 #указываем ip адрес контейнера, для дабл топа,
трипл прихлопа это важно!
restart: unless-stopped
logging:
  driver: "json-file"
  options:
    max-size: "10m"

```

```

    max-file: "5"
    # особая штука, если у вас сингл хоп - убираем этот сервис, он нужен только для
    # правильной маршрутизации других сервисов
    routing:
      container_name: routing
      image: hungcabinet/tun-routing:latest
      cap_add:
        - NET_ADMIN
        - SYS_MODULE
        - SYS_MODULE
      environment:
        TUN: xray-tun0 # название tun интерфейса поднятого в рамках 3x-ui, через
        # который пойдет трафик остальных сервисов
        IPV4_EXCLUDED_ROUTES: |- # ip адреса которые надо исключить из роутинга,
        # важно указать те что есть в моем списке
        127.0.0.0/24
        192.168.1.0/24
        10.0.0.0/8
        172.18.0.0/16
        1.2.3.4/32 # тут важно указать адрес самого сервера
        4.3.2.1/32 # а тут важно указать адрес выходной ноды, чтоб трафик к ней
        # шел напрямую
        1.1.1.1/32 # dns адреса самого сервера, у меня это почти всегда эти 2
        8.8.8.8/32
      network_mode: "service:3x-ui"
      restart: unless-stopped
      depends_on:
        3x-ui:
          condition: service_healthy
      logging:
        driver: "json-file"
        options:
          max-size: "10m"
          max-file: "5"

```

Также в этом докер файле (в конце) нужно указать сеть:

```

networks:
  main:
    driver: bridge
    ipam:
      config:
        - subnet: 172.128.0.0/24 #можете использовать любой адрес, какой вам
        # хочется, но не забывайте менять и в контейнерах

```

После того как сервис поднимется, заходим в веб-панель через SSH-туннель (или через nginx, или еще как вашей душе угодно)

1. Настраиваем входящее хгау подключение как в предыдущем случае. Помним, что в случае использования sing-box клиентов, указывать надо RAW транспорт, безопасность reality и у

пользователей обязательно указывать flow - xtls-rprx-vision. Другие варианты транспорта увы данными клиентами не поддерживаются. Клиенты в разных версиях панели настраиваются по разному... Потому подробно описывать не буду. Заводим нужное количество клиентов, аналогично тому, как делали это в поднятии входной ноды. Рекомендую каждому пользователю (а может и каждому девайсу) делать отдельного пользователя. Для ссылок подписок также нужно менять ip и порт на те, что прокинуты наружу (глобальный ip сервера и порт, на который забинжен ваш внутренний vless порт наружу)

2. Если это дабл хоп, настраиваем tun интерфейс для остальных сервисов. Идём во входящие, создаем подключение

- Протокол tun
- Порт вообще любой, тут это бутафория
- В протоколе указываем имя интерфейса (из докер конфига сервиса routing)
- MTU рекомендую 9000, но можно и 1500 поставить
- Gateway оставляем пустым; на момент написания доки — ни на что не влияло, IP-адрес интерфейсу присвоит контейнер routing
- DNS пишем те же, что указывали в сервисе 3x-ui
- Сохраняем

3. Делаем SOCKS-подключение. Раздел «**Входящие**» → создаём подключение:

- Протокол mixed
- Адрес пишем 172.128.0.2 (или тот, что был указан у сервиса в докер композе)
- Порт пишем любой не занятый другими сервисами (важно! этот порт мы во внешний мир из докера не вытаскиваем)
- Протокол poauth (история локальная, потому тут авторизация не важна)
- Ставим галку UDP, UDP IP пишем тот же что и в основных настройках
- Сохраняем

4. Если у нас дабл-хоп система: **Исходящие** → новое подключение → вкладка JSON → вставляем ссылку от выходной ноды → импорт. На первой вкладке вводим осмысленный тег. Сохраняем. Кнопка «Проверить» должна показать, что трафик между серверами ходит

5. **Настройки** → **Конфигурация XRAY** → **Маршрутизация** — создаём новый маршрут:

- В теги входящих выбираем наши входящие подключения. Да, все три (или больше, если вы создали в хгау другие протоколы)
- Тег исходящего выбираем те, что созданы были в пункте ранее
- Жмём создать
- Перемешаем данное правило повыше, можно в целом на самый верх
- Можно в маршруте который ведёт в direct указать в качестве выходного тега blocked, чтоб все что прошло мимо нашего маршрута падало
- Технически, тут же можно настраивать логику "запросы на русские ip уходят в блок", но пока я опускаю этот момент, так как есть проблемы с данным подходом
- После жмём Сохранить в верху окна и Перезапуск XRAY

ВАЖНО! При обновлениях панели теги входящих при повторных сохранениях подключения могут меняться! После таких манипуляций заходите в маршруты и проверяйте, не слетели ли

теги. Указать входящие теги в маршруте очень важно: если отправить весь трафик во вновь созданный канал — могут быть проблемы с подключением у клиентов. Ловил поведение, при котором обратный трафик к клиенту уходил на второй сервер, а не к клиентам

Поздравляю, базовый конфиг 3x-ui настроен!

wg-easy или VPN с эффектом потери памяти

[Ссылка на проект](#) · [Сайт Amnezia](#) · [Документация Amnezia](#)

Когда нужен: AmneziaWG на входной ноде (при single-hop — можно поднимать как привычно).

Для поднятия AmneziaWG используем wg-easy, а не клиент Amnezia. При дабл-хопе нужна правильная маршрутизация через TUN 3x-ui.

Compose-фрагмент: wg-easy: container_name: wg-easy # latest тег у проекта не всегда стабилен, если выходит стабильная версия, панель выводит об этом информацию image: ghcr.io/wg-easy/wg-easy:15.3.0 volumes: - ./wg-easy/data:/etc/wireguard - /lib/modules:/lib/modules:ro cap_add: - NET_ADMIN - SYS_MODULE environment: - INSECURE=true - EXPERIMENTAL_AWG=true - OVERRIDE_AUTO_AWG=awg - PORT=51821 # это порт веб панели, не подключения. Порт подключения мы настроим позже network_mode: "service:3x-ui" # Если сингл хоп, то можно указать отдельный ip и ту же сеть, что у главного сервиса restart: unless-stopped depends_on: 3x-ui: condition: service_healthy logging: driver: "json-file" options: max-size: "10m" max-file: "5"

> ****Безопасность:**** `INSECURE=true` отключает HTTPS у веб-панели wg-easy — допустимо только за SSH-туннелем. Не выставляйте панель наружу в таком режиме.

После запуска контейнера заходим через SSH-туннель в веб-админку. Проходим простой визард и настраиваем веб панель. В целом ничего сложного, почти всё потом можно будет поменять

Далее логинимся и настраиваем сам сервис. Важно сделать это до создания клиентов, потому как конфиги оно автоматически при изменении параметров обновляет не всегда. Идем в веб панели в администрирование. Также настройки подключения раскиданы по двум разным вкладкам... местами дублируются, но настроить это надо один раз. Так что немного страдаем, но не сильно. Информацию о значении параметров лучше почитать в документации амнезии

1. Общие настройки - как правило ничего не меняем

2. Конфигурация

- Хост - вбиваем глобальный ip адрес вашего сервера
- Порт - ставим порт подключения, его же потом прокидываем в портах сервиса или главного контейнера (в случае дабл хопа)
- Разрешенные IP адреса не трогаем
- DNS - обычно пишу 1.1.1.1 и 8.8.8.8
- MTU - предлагают 1420, а то и 1480, но это сильно влияет на скорость и стабильность... Опытным путём выведено, что лучше поставить 1380, а то и 1320
- Постоянное поддержание соединения - ставим 25, это стандарт
- Количество мусорных пакетов (Js) - ставим не сильно большое, оптимально 4-8.

Напрямую влияет на скорость

- Минимальный размер мусорных пакетов (Jmin) - оптимально от 5 до 15. Напрямую влияет на скорость
- Максимальный размер мусорных пакетов (Jmax) - оптимально от 70 до 110.

Напрямую влияет на скорость

- Мусорные пакеты I1-I5 – можно и не заполнять, но рекомендую заполнить хотя бы I1. Не все клиенты нормально работают со всеми тегами. Потому тут экспериментируем и читаем про данные параметры в документации Amnezia (пункт AmneziaWG).

Рекомендуется использовать не больше двух тегов. Не влияет напрямую на скорость

- Сохраняем

3. Сетевой интерфейс (рекомендуется открыть в отдельной вкладке, так как некоторые параметры дублируются)

- MTU - ставим такой же как на предыдущей вкладке
- Порт - такой же как на предыдущей вкладке
- Устройство - оставляем eth0
- Jc, Jmin, Jmax, I1-I5 - полностью копируем с предыдущей вкладки
- S1, S2 - заполняем по документации амнезии или по подсказке. Чем больше, тем сильнее влияет на скорость. Чем меньше, тем менее "спрятан" ваш трафик
- S3, S4 - можно не заполнять, тогда протокол будет считаться версией 1.5, а не 2.0. Добавляя их, увеличиваем обфускацию, и при этом снижаем скорость подключения
- H1 - H4 - заполняем по подсказке. Не влияет на прямую на скорость
- Если хочется, можем тут поменять CIDR (адрес сети, которую будет выдавать клиентам). В большинстве случаев это не нужно
- Сохраняем

4. Создаём клиентов. Если конфиг большой, QR-код может не генерироваться – это нормально. Конфиги копируем вручную.

****Что отдать пользователю:**** конфиг AmneziaWG (файл) или ссылку через портал / подписку hc-box.

NaïveProxy или как наивно недооценивать этот протокол

[Ссылка на проект](https://github.com/klzgrad/naiveproxy)

****Когда нужен:**** максимальная маскировка под HTTPS; не подходит для UDP-игр и звонков.

Итак. Этот протокол максимально приблизился к тому, чтобы притвориться веб-трафиком: сервер – Caddy, клиент – урезанный Chrome. Веб-панели нет, только конфигами.

Поддерживается два протокола: https (tcp) и quic (udp). Чисто технически можно использовать что тот, что тот... Но с quic и вообще поддержкой udp трафика у данного протокола всё достаточно грустно. Потому он скорее подходит для веб сёрфинга, ютубчика и прочих вещей... но не для игр или звонков с использованием udp

Для того чтоб развернуть данный протокол вам понадобится своё доменное имя прикрепленное к данному ip. Вам нужен сайт заглушка. В папке `naiveproxy/www/site` лежит пример. Можно попросить любую нейросеть сгенерировать сайт заглушку любой сложности. Но для простоты рекомендую попросить поменять html и css под какую то

тематику не меняя работы js скрипта. Всё сделано так, чтоб маломальски походить на нормальный сайт

Если всё это было проделано - правим под себя конфиг `naiveproxy/Caddyfile`. Нужно заменить `my.site.example.com` на название вашего сайта и указать столько секций

```
forward_proxy {
    basic_auth user1 password1
    hide_ip
    hide_via
    probe_resistance as2jhfawasvevaasdaafaasf35.com
}
```

сколько у вас будет пользователей. Для каждого юзера отдельная секция. После создаем сервис в компоуз файле:

```
naiveproxy: container_name: naiveproxy image: pocat/naiveproxy:latest-uot restart: unless-stopped
network_mode: "service:3x-ui" depends_on: 3x-ui: condition: service_healthy volumes: -
/etc/hungcabinet/naiveproxy/www/site:/var/www/site - /etc/hungcabinet/naiveproxy/logs:/var/log/caddy -
/etc/hungcabinet/naiveproxy:/etc/naiveproxy - /etc/hungcabinet/naiveproxy/data:/root/.local/share/caddy
environment: - PATH=/etc/naiveproxy/Caddyfile logging: driver: "json-file" options: max-size: "10m" max-file:
"5"
```

Если всё сделано правильно, по доменному имени будет доступен сайт-заглушка.

****Что отдать пользователю:**** IP, домен, логин и пароль из `Caddyfile` (или ссылку через портал).

Mieru или как хороший протокол чуть было не прошёл мимо нас

[Страница проекта](https://github.com/enfein/mieru) · [mihomo]
 (https://wiki.metacubex.one/ru/) · [конфиг mieru-слушателя]
 (https://wiki.metacubex.one/ru/config/inbound/listeners/mieru/)

****Когда нужен:**** перспективный малозаметный протокол; сервер через `mihomo`, в дабл-хопе — через SOCKS inbound 3x-ui.

Конфиг самого `mihomo` сервиса выполняющего роль сервера лежит в `hungcabinet/mihomo`. Для запуска сервиса используем следующий компоуз сервис

```

mihomo: image: metacubex/mihomo:latest container_name: mihomo restart: unless-stopped ports: -
"2012:2012/udp" #mieru udp cap_add: - NET_ADMIN - NET_RAW - SYS_MODULE volumes: -
./mihomo/config.yaml:/root/.config/mihomo/config.yaml:ro networks: main: ipv4_address: 172.128.0.3
depends_on: 3x-ui: condition: service_healthy logging: driver: "json-file" options: max-size: "10m" max-file: "5"

```

У клиента и сервера очень важна синхронизация времени! Клиент и сервер должны синхронизировать его по NTP иначе подключение не произойдёт. Также у клиента и сервера есть возможность обфусцировать трафик. Обфускация с двух сторон может быть разной, что увеличивает энтропию трафика. Настраивается это через `traffic-pattern` в конфиге. Полное описание свойств и как настраивать можно прочитать [тут] (<https://github.com/enfein/mieru/blob/main/docs/traffic-pattern.md>)

Для создания base64 строки для конфига mihomo придется скачать родной клиент mieru на ваш ПК, а не сервер. Создать json конфиг (можно взять прям из примера) и настроить в нём pattern так, как нужно вам

```

{ "profiles": [ { "profileName": "default", "user": { "name": "ducaiguozai", "password": "xijinping" }, "servers": [ {
"ipAddress": "12.34.56.78", "portBindings": [ { "portRange": "2012-2022", "protocol": "TCP" } ] },
"trafficPattern": { "unlockAll": false, "tcpFragment": { "enable": true, "maxSleepMs": 10 }, "nonce": { "type":
"NONCE_TYPE_PRINTABLE", "applyToAllUDPPacket": true, "minLen": 6, "maxLen": 8 } } ], "activeProfile":
"default", "rpcPort": 8964, "socks5Port": 1080 }

```

> **Безопасность:** логин и пароль в примере выше — шутка для лаборатории. Замените на свои перед продом.

После чего выполнить следующие манипуляции в командной строке:

```
<путь до mieru> apply config <путь до конфига с паттерном> <путь до mieru> export traffic-pattern
```

Последняя команда выдаст строку вроде `EAAaBAgBEAoiCAgBEAEYBiAI` — её вставляем в конфиг mihomo.

Что отдать пользователю: конфиг mieru (когда появится поддержка в hc-box) или параметры из mihomo-конфига.

Telemt, если Telegram не едет

[Страница проекта](<https://github.com/telemt/telemt>) · [панель] (https://github.com/amirotin/telemt_panel)

****Когда нужен:**** MTPROTO-прокси для Telegram. Сейчас активно банится – чаще проще VPN + локальный SOCKS.

> **ВАЖНО!** Контейнер панели я собираю образ сам, так как автор проекта пока не делает это. Потому он временами будет отставать от актуальной версии

Конфигурация телемт достаточно обширная и лучше почитать доку. Я представил минимальный пример, для того чтоб оно заработало в том сетапе, который предлагается. Для работы панели важно сохранить секцию `[server.api]`. Сами конфиги лежат в `telemt/config.toml` и `telemt-panel/config.toml`. С генерацией секретов отлично справляется сайт [guidgenerator.com] (<https://www.guidgenerator.com/>) со всеми снятыми галочками. Также юзеров можно заводить через веб панель

Также, если мы заняли 443 порт naiveпроху, то указать надо какой то другой порт. В секции `[general.links]` нужно указать свой публичный ip (глобальный ip сервера), чтоб в панели можно было нормально копировать ссылки. Как сгенерировать хэш пароля и jwt для веб панели подробно расписано в рамках документации к веб панели... остальные данные в конфиге веб панели можно не менять

```
telemt: image: ghcr.io/telemt/telemt:latest container_name: telemt restart: unless-stopped working_dir:
/etc/telemt # вот это прям важно, в противном случае процесс не сможет менять конфиг и панель не
будет работать на редактирование user: root:root volumes: - ./telemt:/etc/telemt:rw environment: -
RUST_LOG=info healthcheck: test: [ "CMD", "/app/telemt", "healthcheck", "/etc/telemt/config.toml", "--mode",
"liveness" ] interval: 30s timeout: 5s retries: 3 start_period: 20s network_mode: "service:3x-ui" depends_on: 3x-
ui: condition: service_healthy security_opt: - no-new-privileges:true ulimits: nofile: soft: 65536 hard: 262144
logging: driver: "json-file" options: max-size: "10m" max-file: "5" telemt-panel: image: hungcabinet/telemt-
panel:latest container_name: telemt-panel volumes: - ./telemt-panel/config.toml:/etc/telemt-
panel/config.toml:ro - ./telemt/config.toml:/etc/telemt/config.toml:ro restart: unless-stopped network_mode:
"service:3x-ui" depends_on: - telemt logging: driver: "json-file" options: max-size: "10m" max-file: "5"
```

****Что отдать пользователю:**** ссылку `tg://proxy?...` из панели Telemt (секция на портале «Telegram Proxies»).

> ****Важно!**** MTPROTO сейчас активно банится – зачастую проще VPN вместо прокси или «дурилок» типа zapret/byedpi.

Генерируем конфиги и паблишим их на сервере

Для генерации конфигов и паблишинга на веб сервере можно использовать утилиту: [hc-configs](<https://github.com/hungcabinet/hc-configs>)